```python
def merge_intervals(intervals):
    if not intervals:
        return []

    # Step 1: Sort intervals by start time
    intervals.sort(key=lambda x: x[0])

    # Step 2: Initialize result with first interval
    merged = [intervals[0]]

    # Step 3: Iterate and merge
    for current in intervals[1:]:
        last = merged[-1]
        if current[0] <= last[1]:
            last[1] = max(last[1], current[1])
        else:
            merged.append(current)

    return merged

# Test with example
intervals = [[1,3], [2,6], [8,10]]
print(merge_intervals(intervals))
```

```
[[1, 6], [8, 10]]
```

```python
from collections import defaultdict

def group_anagrams(strs):
    d = defaultdict(list)

    for word in strs:
        key = ''.join(sorted(word))   # sort letters
        d[key].append(word)

    return list(d.values())


# Example
print(group_anagrams(["eat","tea","tan","ate","nat","bat"]))
```

```
[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

```python
from collections import deque

def max_sliding_window(nums, k):
    dq = deque()
    result = []

    for i in range(len(nums)):
        # Remove out-of-window elements
        while dq and dq[0] < i - k + 1:
            dq.popleft()

        # Remove smaller elements
        while dq and nums[dq[-1]] < nums[i]:
            dq.pop()

        dq.append(i)

        # Add result
        if i >= k - 1:
            result.append(nums[dq[0]])

    return result


# Example
print(max_sliding_window([1,3,-1,-3,5,3,6,7], 3))
```

```
[3, 3, 5, 5, 6, 7]
```

```python
def is_valid(s):
    stack = []
    mapping = {')':'(', '}':'{', ']':'['}

    for ch in s:
        # If closing bracket
        if ch in mapping:
            if not stack or stack.pop() != mapping[ch]:
                return False
        else:
            stack.append(ch)

    return len(stack) == 0


# Example
print(is_valid("({[]})"))
print(is_valid("({[})"))
```

```
True
False
```

```python
def length_of_longest_substring(s: str) -> int:
    last_seen = {}
    left = 0
    max_len = 0

    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1
        last_seen[ch] = right
        max_len = max(max_len, right - left + 1)

    return max_len

# Example
print(length_of_longest_substring("abcabcbb"))
print(length_of_longest_substring(""))
print(length_of_longest_substring("bbbbb"))
print(length_of_longest_substring("pwwkew"))
```

```
3
0
1
3
```

```python
def twoSum(nums, target):
    hash_map = {}

    for i, num in enumerate(nums):
        complement = target - num
        if complement in hash_map:
            return [hash_map[complement], i]
        hash_map[num] = i

    return []

# Test
nums = [2, 7, 11, 15]
target = 9
print(twoSum(nums, target))
```

```
[0, 1]
```

```python
def product_except_self(nums):
    n = len(nums)
    result = [1] * n

    # Prefix product
    prefix = 1
    for i in range(n):
        result[i] = prefix
        prefix *= nums[i]

    # Suffix product
    suffix = 1
    for i in range(n-1, -1, -1):
        result[i] *= suffix
        suffix *= nums[i]

    return result


# Example
nums = [1, 2, 3, 4]
print(product_except_self(nums))
```

```
[24, 12, 8, 6]
```

```python
def length_of_longest_substring(s: str) -> int:
    last_seen = {}
    left = 0
    max_len = 0

    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1
        last_seen[ch] = right
        max_len = max(max_len, right - left + 1)

    return max_len

# Example
print(length_of_longest_substring("abcabcbb"))
print(length_of_longest_substring(""))
print(length_of_longest_substring("bbbbb"))
print(length_of_longest_substring("pwwkew"))
```

```
3
0
1
3
```

```python
def product_except_self(nums):
    n = len(nums)
    result = [1] * n

    # Prefix product
    prefix = 1
    for i in range(n):
        result[i] = prefix
        prefix *= nums[i]

    # Suffix product
    suffix = 1
    for i in range(n-1, -1, -1):
        result[i] *= suffix
        suffix *= nums[i]

    return result


# Example
nums = [1, 2, 3, 4]
print(product_except_self(nums))
```

```
[24, 12, 8, 6]
```